STUDYVAULT

DIGITAL STUDY MATERIAL MANAGEMENT SYSTEM

PROJECT REPORT

Submitted in partial fulfillment of the requirements for

T.Y. B.Sc. Computer Science

Submitted By: _____

Roll No.: _____

Guided By: _____

Department of Computer Science

College Name: _____

Academic Year: _____

INDEX

Sr. No.	Title	Signature of the Instructor
1	1.1 Problem Identification 1.2 Problem Statement / Definition 1.3 Purpose, Objectives and Goals 1.4 Feasibility Study 1.5 Project Scope and Limitations	
2	**Requirement Specification** 2.1 Hardware / Software Requirements 2.2 Technical Requirements 2.3 Functional Requirements 2.4 Data, Performance and Security Requirements	
3	**Database Design** 3.1 Identify End Users 3.2 Entities and Attributes through ER Model 3.3 Tables, Fields and Relationships	
4	**System Design** (to be added separately later)	
5	**Input and Output Screen Layout** (to be added separately later)	
6	**Bibliography / References**	

1. PROBLEM IDENTIFICATION

1.1 Problem Identification

Students often receive study materials from many sources, such as classroom groups, websites, messaging applications and shared drives. As a result, notes and documents may become scattered, difficult to search, and hard to organize by course, semester and subject. StudyVault is planned as a centralized web-based system for managing and accessing academic study materials.

1.2 Problem Statement / Definition

To design and develop a secure and user-friendly web application that allows an administrator to manage courses, semesters, subjects and study materials, while students can search, view and download relevant materials in an organized manner.

1.3 Purpose, Objectives and Goals

- Provide one centralized location for academic notes and documents.
- Organize materials according to course, semester and subject.
- Allow authorized administrators to upload, update and delete materials.
- Allow students to browse, search, view and download available resources.
- Reduce the time required to locate study materials.
- Provide login authentication and role-based access.

1.4 Feasibility Study

Technical Feasibility: The system can be built using HTML, CSS and JavaScript for the interface, Python Flask for server-side processing, and PostgreSQL for data storage. These technologies are suitable for a student project and have broad documentation support.

Economic Feasibility: The main technologies are open source, which keeps software costs low. Development can be carried out on an existing computer.

Operational Feasibility: Simple forms, menus and search options will help students and administrators use the application with basic computer knowledge.

Schedule Feasibility: Work can be completed in stages: requirement analysis, database design, interface development, backend development, testing and documentation.

1.5 Project Scope and Limitations

Scope: Course-, semester- and subject-based material organization; administrator management of study materials; student login; searching, viewing and downloading documents; and basic user management.

Limitations: The initial version requires an internet or local network connection. The system depends on the quality of uploaded materials. Features such as a dedicated mobile application, cloud-scale storage and automatic plagiarism checking may be considered in future versions.

2. REQUIREMENT SPECIFICATION

2.1 System Requirements – Hardware and Software

Hardware Requirements (minimum suggested)

- Processor: Dual-core processor or equivalent.
- RAM: 4 GB minimum; 8 GB recommended.
- Storage: At least 2 GB free for development tools and project files, plus space for uploaded documents.
- Display: Standard laptop or desktop display.
- Network: Internet or local network for accessing the application.

Software Requirements

- Operating System: Linux (development environment).
- Frontend: HTML5, CSS3 and JavaScript.
- Backend: Python 3 with Flask.
- Database: PostgreSQL.
- Browser: A recent version of Chrome, Firefox or another modern browser.
- Editor / Tools: Visual Studio Code and a PostgreSQL client such as pgAdmin or psql.

2.2 Technical Requirements

Programming languages: Python, HTML, CSS and JavaScript. Framework: Flask. Database Management System: PostgreSQL. The application follows a client-server structure. The browser sends requests to the Flask application; Flask processes requests and reads or updates data in PostgreSQL; then the response is displayed in the browser.

2.3 Functional Requirements

- The system shall allow an administrator to log in.
- The administrator shall be able to add, edit and delete courses and subjects.
- The administrator shall be able to upload, update and delete study materials.
- Students shall be able to register or log in, according to the configured registration policy.
- Students shall be able to browse materials by course, semester and subject.
- Students shall be able to search for materials and view file details.
- Students shall be able to download permitted study materials.
- The system shall validate required fields and display clear messages.

2.4 Data, Performance and Security Requirements

Data Requirements: Store user information, course and subject details, and study-material metadata such as title, description, file name, file path and upload date.

Performance Requirements: Pages and searches should respond promptly under normal project usage. File upload size limits should be configured.

Security Requirements: Store passwords as secure hashes, validate input on the server, use safe database queries, restrict administrator actions to authorized users, and validate uploaded file types and sizes. Regular backups are recommended.

3. DATABASE DESIGN

3.1 Identify End Users of the System

1. Administrator: Manages users, courses, subjects and study materials.
2. Student: Browses, searches, views and downloads study materials.

3.2 Identify Entities and Attributes through ER Model

- **User:** user_id (Primary Key), full_name, email, password_hash, role, created_at.
- **Course:** course_id (Primary Key), course_name, course_code, description.
- **Subject:** subject_id (Primary Key), course_id (Foreign Key), semester, subject_name, subject_code.
- **StudyMaterial:** material_id (Primary Key), subject_id (Foreign Key), uploaded_by (Foreign Key), title, description, file_name, file_path, file_type, uploaded_at.
- **Download (optional):** download_id (Primary Key), material_id (Foreign Key), user_id (Foreign Key), downloaded_at.

Relationships: One course can contain many subjects. One subject can have many study materials. One administrator can upload many materials. One student can download many materials, and each material can have many download records.

3.3 Identify Tables, Fields and Relationships

users: user_id SERIAL PRIMARY KEY; full_name VARCHAR(100); email VARCHAR(150) UNIQUE; password_hash TEXT; role VARCHAR(20); created_at TIMESTAMP.

courses: course_id SERIAL PRIMARY KEY; course_name VARCHAR(120); course_code VARCHAR(30); description TEXT.

subjects: subject_id SERIAL PRIMARY KEY; course_id INTEGER REFERENCES courses(course_id); semester INTEGER; subject_name VARCHAR(120); subject_code VARCHAR(30).

study_materials: material_id SERIAL PRIMARY KEY; subject_id INTEGER REFERENCES subjects(subject_id); uploaded_by INTEGER REFERENCES users(user_id); title VARCHAR(200); description TEXT; file_name VARCHAR(255); file_path TEXT; file_type VARCHAR(30); uploaded_at TIMESTAMP.

downloads (optional): download_id SERIAL PRIMARY KEY; material_id INTEGER REFERENCES study_materials(material_id); user_id INTEGER REFERENCES users(user_id); downloaded_at TIMESTAMP.

ER Model – Relationship Overview

COURSES (1) ■■■■■ (Many) SUBJECTS
SUBJECTS (1) ■■■■■ (Many) STUDY_MATERIALS
USERS (Administrator) (1) ■■■■■ (Many) STUDY_MATERIALS
USERS (Student) (1) ■■■■■ (Many) DOWNLOADS (Many) ■■■■■ (1) STUDY_MATERIALS

Note: The administrator can be stored in the users table with role='admin'; a separate admin table is not required unless the implementation needs it.

6. BIBLIOGRAPHY / REFERENCES

- Python Documentation – https://docs.python.org/3/
- Flask Documentation – https://flask.palletsprojects.com/
- PostgreSQL Documentation – https://www.postgresql.org/docs/
- MDN Web Docs (HTML, CSS and JavaScript) – https://developer.mozilla.org/
- Visual Studio Code Documentation – https://code.visualstudio.com/docs